arXiv:2309.03450v1 [cs.CL] 7 Sep 2023

XGen-7B Technical Report

Erik Nijkamp[*], **Tian Xie**[*], **Hiroaki Hayashi**[*], **Bo Pang**[*], **Congying Xia**[*], **Chen Xing**

Jesse Vig, Semih Yavuz, Philippe Laban, Ben Krause, Senthil Purushwalkam, Tong Niu

Wojciech Kryściński, Lidiya Murakhovs'ka, Prafulla Kumar Choubey, Alex Fabbri

Ye Liu, Rui Meng, Lifu Tu, Meghana Bhat, Chien-Sheng Wu, Silvio Savarese

Yingbo Zhou[†], **Shafiq Joty**[†], **Caiming Xiong**[†]

Salesforce Research

Abstract

Large Language Models (LLMs) have become ubiquitous across various domains, transforming the way we interact with information and conduct research. However, most high-performing LLMs remain confined behind proprietary walls, hindering scientific progress. Most open-source LLMs, on the other hand, are limited in their ability to support longer sequence lengths, which is a key requirement for many tasks that require inference over an input context. To address this, we have trained XGen-7B, a series of 7B parameter models on up to 8K sequence length for up to 1.5T tokens. We have also finetuned the XGen-7B models on public-domain instructional data, creating their instruction-tuned counterparts (XGen-7B-Inst). We open-source our models for both research advancements and commercial applications. Our evaluation on standard benchmarks shows that XGen-7B models achieve comparable or better results when compared with state-of-the-art open-source LLMs. Our targeted evaluation on long sequence modeling tasks shows the benefits of our 8K-sequence models over 2K-sequence open-source LLMs.

1 Introduction

Large language models (LLMs) have shown impressive capabilities to generate text, translate languages, write code, answer questions, solve math problems, predict actions, and many more. Interestingly, they can perform these tasks from textual instructions and/or observing a few demonstrations [2]. Crucial to their success are two main ingredients: (a) *model scale* which defines the model's capacity; and (b) *instruction tuning*, which aims to align the model to follow user instructions [25].

While the proliferation of LLMs has enhanced numerous applications, a significant number of high-performing models remain proprietary, impeding the progress of scientific exploration. Recent work [14] on model scaling has shown that for a given compute budget, the best performances are not necessarily achieved by the largest models, but by smaller models trained on more data (measured by the number of tokens). A smaller model is also generally preferred for inference efficiency during serving including on mobile devices.

As LLMs become ubiquitous, their applications to long sequences have been a key focus [33, 30], especially for applications like writing code, summarizing text (potentially interleaved with other

[*]indicates lead authors, [†]indicates lead project coordinators and corresponding authors.

Table 1: High-level summary of the XGen-7B models.

Model	Description
XGen-7B-4K	Pre-train for 800B tokens with a sequence length of 2K tokens first, then for another 400B tokens (total 1.2T tokens) with 4K tokens.
XGen-7B-8K	Initialize with XGen-7B-4K-base and further train for 300B more tokens (total 1.5T tokens) with 8K sequence length.
XGen-7B-Inst$_{\text{wizardLM}}$	Supervised fine-tuning of XGen-7B-8K on the recently released WizardLM-196K [39] instruction data.
XGen-7B-Inst$_{\text{general}}$	Supervised fine-tuning of XGen-7B-8K on general public domain instruction data including OAsst[5], Baize [40], Dolly2 [10], ShareGPT and SCROLLS [30].

data sources like tables and images), and predicting protein sequences, which require the model to effectively consider long distance structural dependencies. A large context allows a pre-trained LLM to look at customer data (e.g., documents the LLM did not use in training) and responds to useful information seeking queries. Yet, most open-source LLMs (e.g., LLaMA [34], MPT[2], Falcon[3]) have been trained with a maximum of 2K token sequence length, which is a key limitation in modeling long sequences. Inference time solutions such as ALiBi [26] have yet to be tested properly for larger models (e.g. MPT-7B-StoryWriter-65k+).

To address the above limitation, in light of the scaling properties and serving efficiency, we train a series of 7B LLMs named XGen-7B with standard dense attention on up to 8K sequence length for up to 1.5T tokens. We also finetune the XGen-7B models on public-domain instructional data, creating their instruction-tuned counterparts (XGen-7B-Inst). We open-source our models for both research advancements and commercial applications. Table 1 summarizes our released models.[4].

Our evaluation of XGen-7B-8K on standard benchmarks for evaluating base pre-trained models shows that it achieves comparable or better results when compared with state-of-the-art open-source LLMs. It also achieves good results on Python code generation tasks. Our instruction-tuned models also show impressive results on the recently proposed AlpacaEval [16] and MTBench [43] benchmarks, often outperforming models of similar sizes (e.g., WizardLM-7B, MPT-7B) and even larger ones (e.g., Falcon-40B-instruct, Alpaca-13B). Furthermore, our targeted evaluation on long sequence modeling tasks show benefits of our 8K-sequence models over 2K-sequence open-source LLMs.

2 Pre-training Data

Our pre-training dataset is a mixture of data from several public sources, reported in Table 2. We employ a two-stage training strategy, where each stage uses a different data mixture, as shown in Table 3.

Natural language data for stage 1. Natural language data is a mixture of publicly available data. We made an effort to improve safety and diversity of the data.

Code data for stage 1. We use the GitHub subset from the recently released RedPajama dataset [9]. We also added Apex code data to enhance our model's proficiency in Apex code generation. Apex is a widely used object-oriented programming language in Salesforce products.

BigCode Starcoder data for stage 2. We use all the 86 programming languages from the Starcoder [15] data, preserving the original weight of each. Subsequently, we further filter the data according to a stronger permissive license guideline.

Tokenizer. We tokenize the data with the byte pair encoding (BPE) algorithm [29], utilizing OpenAI's tiktoken tool, with GPT-2 serving as the base tokenizer. Additionally, we incorporate supplementary special tokens as outlined in the Starcoder paper [15], along with consecutive white-spaces and tabs with the goal of aiding code generation.

[2] `https://www.mosaicml.com/blog/mpt-7b`

[3] `https://falconllm.tii.ae/`

[4] `https://github.com/salesforce/XGen`

Table 2: Data mixtures used for pre-training stage 1. For each subset of the data, we report the effective number of tokens, and its sampling proportion.

Dataset	Tokens (B)	Sampling prop. (%)
Natural language data	1309.99	95.31
Code data	64.53	4.69
Total	1374.52	100

Table 3: Data mixtures used for pre-training stage 2.

Dataset	Tokens (B)	Sampling prop. (%)
Data from stage 1	55	50
BigCode Starcoder data	55	50
Total	110	100

Constructing sequences of different lengths. During the pre-training stage 1, there are 3 substages, each with varying sequence lengths: 2K, 4K, and 8K tokens. To ensure data integrity and prevent potential distributional shifts, we shuffle the data uniformly, and split the shuffled data into 3 big chunks for the 3 substages. We construct the training sequences by concatenating or splitting the original text documents into the target sequence lengths. When two different documents are concatenated, an `<|endoftext|>` token is added between them. We exclude short documents that contain less than 100 tokens after tokenization. We then shuffle the constructed training sequences uniformly in each big chunk. The data for pre-training stage 2 (50% stage 1 data and 50% Starcoder data) only has training sequences with a length of 8k tokens.

3 Training Details

The XGen-7B models are trained with our library JaxFormer [22], which facilitates efficient training of LLMs under both data and model parallelism optimized for TPU-v4 hardware. The training recipe and model architecture follow LLaMA [34], while we conduct two additional explorations.

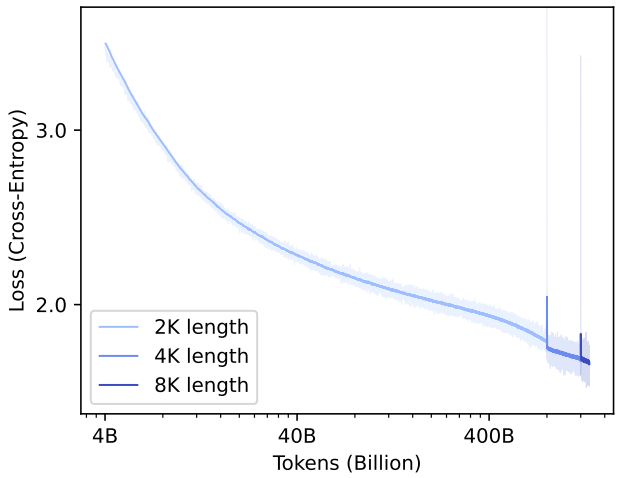

Figure 1: Cross-entropy over training time. The model is pre-trained in three stages with a step-wise increase of the sequence length from 2K to 4K to 8K tokens. Note, the pre-training does not suffer from any loss spikes. The spikes at the transitions in sequence lengths are expected as the model adjusts to positional encodings of increased length. The drop in perplexity from 2K to 4K is expected as uncertainty decreases over sequence length for long sequences.

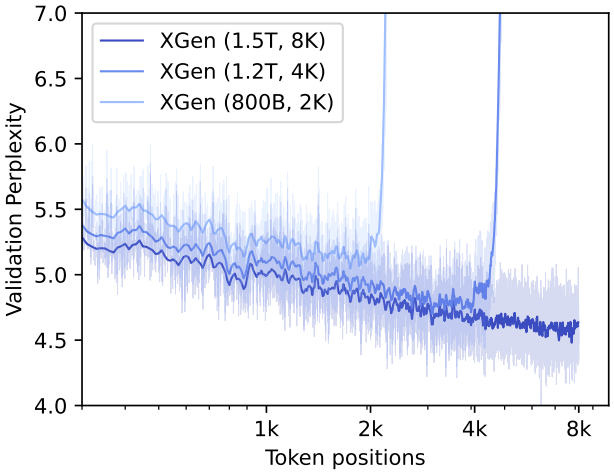

Figure 2: Perplexity over sequence length. If a model can utilize the information encoded in long sequences, then, in expectation, the perplexity should decrease over the length of such sequences. That is, the information contained in previous tokens increases the certainty of the next token prediction, which can be observed in the figure. Note, the perplexity of the 8K over the 2K model is generally lower as the model has been trained for an additional 700B tokens.

First, we investigate the occurrence of so-called "loss spikes" [6, 35, 20] during training, that is, the loss suddenly explodes temporarily while the root cause for these spikes is unknown. Second, the XGen-7B models support sequence lengths of up to 8,192 tokens (rather than the common 2,048) for which we introduce stage-wise training.

Recipe. The model architecture follows LLaMA with exact numerical compatibility to ease adoption in third-party frameworks. The hyperparameters closely follow LLaMA-7B [34] with the following alterations: (1) The token budget has been increased from 1.0T to 1.5T tokens, (2) the training is performed stage-wise to increase the sequence length from 2K to 4K to 8K, (3) the vocabulary size has been increased from 32,000 to 51,200 tokens. The training loop was implemented in JAX with Haiku for which the entire computation is under FP32 numerical precision, except for matmul in BF16.

Loss spikes. As models are scaled to larger sizes, the training itself is increasingly sensitive to instabilities, which cause poor model performance, if not addressed carefully [6, 35, 20]. In our exploration, we have gathered evidence for several factors, which individually contribute to unstable training. These preliminary findings include "sequential over parallel self-attention circuits", "swish-GLU over GeLU", "RMS-Norm over Layer-norm". Specifically, widely used parallel circuits [6, 35, 23], which parallelize the computation of self-attention and feed-forward may affect the stability of training, at least in our specific setting. As adopted in [34], the combination of activation normalization in the form of RMS-Norm [42], sequential self-attention and swish-GLU [31] appears to be numerically highly robust, while not optimal in terms of computational efficiency.

Sequence length. Training with longer sequences is computationally unproportionally costly as the complexity of self-attention is quadratic, that is, the training process is slow. To mitigate slow training, we introduce training in stages with increasing sequence length. First, 800B tokens with sequence length of 2K tokens are observed, then 400B tokens with 4K, finally, 300B tokens with 8K length. Figure 1 shows the cross-entropy over training steps for this stage-wise training. We verify the adaptation to longer sequences by computing the average perplexity at each token position on a held-out validation set containing documents of 8K sequence length or above. If the model successfully learns to utilize the full sequence, we would expect the perplexity to decrease over sequence length, as previous tokens carry information for the next to-be-predicted token. That is, for a long sentence, the more context in the form of previous words is provided, the easier it becomes to

guess the next word. Figure 2 indeed demonstrates that XGen-7B at each stage successfully learns to utilize longer contexts, up to 8K sequence length.

4 Instructional Tuning

To demonstrate the language understanding and generation capability of XGen-7B, we perform instruction-tuning of the base LLM and evaluate the instruct-tuned models.

Instruction Data. The key to instruction tuning is the *instructional data* that is used to align the model to follow user instructions, while being harmless. Developing proprietary models like GPT-4 [24] and Bard [21] involves significant annotation efforts for collecting such data. Early open-source instruction tuned models [18] leverage academic datasets by transforming them into instructional formats with human-written prompt templates [37, 28, 36]. Despite the large amount of data curated in this way, instruction following capacity of these models falls behind proprietary models as the task distributions covered by these academic benchmarks do not match the real use cases of LLMs [25].

Therefore, more recent open-source models [43, 39] utilize ChatGPT- or GPT4-synthesized data, e.g., human-written prompts with GPT-generated responses or GPT-generated prompts and responses. This distillation process helps to close the gap to the proprietary models. Some examples of these datasets are Alpaca [32], ShareGPT [6], Baize [40], GPTeacher [7], and WizardLM [39].

For our experiments, we finetune XGen-7B in two data settings:

- XGen-7B-Inst$_{\text{wizardLM}}$: For this setting, we use WizardLM [39], which is one of the most recent instruction datasets. It is created by prompting GPT-4 to rewrite existing instructions from Alpaca [32] to make them more complex. Finetuning LLaMA models on this dataset has demonstrated high performance in several benchmarks especially for complex instructions. We use the WizardLM-196K collection for finetuning XGen-7B. This setting allows us to compare with the WizardLM-7b model [39], which is based on LLaMA-7b and uses the same source of instructions.
- XGen-7B-Inst$_{\text{general}}$: In another setting, we use general public domain instruction data that includes OAsst[8], Baize [40], Dolly2 [10] and ShareGPT. To measure the impact of long contexts, we also include examples from the long sequence NLP benchmark, SCROLLS [30]. We sample about 1,500 examples from each of the following datasets from SCROLLS: GovReport, SummScreenFD, QMSum, NarrativeQA, Qasper and QuALITY. We sample these examples such that each contains at least 4000 tokens. This setting is intended to give us a sense on the model's general capability in following instructions for long-sequence tasks.

Finetuning Details. We use Adam with $\beta_1 = 0.9$ and $\beta_2 = 0.99$, cosine decay for learning rate down to 10% of an initial value 2×10^{-5}, a batch size of 128, and a sequence length of 8,192 tokens. Each data instance is formatted as a single-turn or multi-turn conversation between Human and Assistant. In particular, it follows the format:

```
### Human: {prompt} ### Assistant: {response}
```

Our training objective is causal language modeling and the loss for `prompt` is masked out, thus only the gradients for `response` tokens are backpropagated. We train our models for 3 epochs.

5 Evaluation

5.1 Base Model Evaluation

5.1.1 Standard NLP Benchmarks

We first consider Massive Multitask Language Understanding benchmark [13], which is more recent and less susceptible to data contamination as reported in recent studies (see page 32 of GPT-4 technical report [24] and a related discussion [9]). The benchmark has been widely adopted for held-out

[6] `https://huggingface.co/datasets/anon8231489123/ShareGPT_Vicuna_unfiltered`

[7] `https://github.com/teknium1/GPTeacher`

[8] `https://huggingface.co/datasets/OpenAssistant/oasst1`

[9] https://hitz-zentroa.github.io/lm-contamination/blog/?ref=blog.salesforceairesearch.com

Table 4: Five-shot results (accuracy) on Massive Multitask Language Understanding (MMLU).

Models	Humanities	STEM	Social Sciences	Other	Weighted average
XGen-7B	33.8	30.7	40.0	41.5	36.3
LLaMA-7B	33.9	30.6	38.2	38.2	35.1
OpenLLaMA-7B	28.1	28.5	31.2	32.8	29.9
Falcon-7B	26.5	25.4	29.2	26.8	26.9
MPT-7B	25.9	26.2	26.9	28.1	26.7
Redpajama-7B	26.1	25.2	27.4	26.7	26.3
Cerebras-GPT-13B	26.1	26.5	25.8	26.6	26.2
Dolly-v2-12B	26.9	25.7	25.3	26.5	26.2
OPT-13B	26.2	24.3	23.4	26.0	25.1
GPT-J-6B	25.9	24.0	24.0	25.8	25.1

Table 5: Zero-shot accuracy on Massive Multitask Language Understanding (MMLU).

Models	Humanities	STEM	Social Sciences	Other	Weighted average
XGen-7B	31.4	27.8	32.1	37.2	32.1
LLaMA-7B	32.3	27.1	31.3	36.8	32.0
OpenLLaMA-7B	28.0	27.6	28.9	30.1	28.6
MPT-7B	27.4	25.2	26.0	30.7	27.4
Redpajama-7B	27.5	25.5	24.2	25.0	25.8
GPT-J-6B	25.3	24.5	25.5	27.6	25.7
Dolly-v2-12B	26.2	26.0	24.0	24.9	25.4
Cerebras-GPT-13B	24.3	25.0	23.0	26.0	24.6
OPT-13B	26.3	23.3	23.6	23.6	24.4
Falcon-7B	24.8	21.7	24.0	24.4	23.9

evaluation. Recently, however, inconsistencies in reporting MMLU scores have been reported, which resulted in wrong rankings in Hugging Face's Open LLM leaderboard. In our work, we follow the original MMLU standard, which is consistent with the published results (i.e., in LLaMA).

MMLU consists of multiple choice questions covering various domains of knowledge, including humanities, STEM and social sciences. To assess our models' performance, we conduct evaluations in both the five- and zero-shot settings, utilizing the sample questions from the benchmark. The results for five-shot MMLU are reported in Table 4, and the results for zero-shot MMLU are reported in Table 5. For both settings, XGen-7B achieves the best results among the baselines in most categories, as well as in weighted average.

We also report general zero-shot results on other standard NLP benchmarks that involve common sense reasoning and QA: ARC challenge [8], HellaSwag [41], Winogrande [27], TruthfulQA [17], BoolQ [7], PiQA [1], and OpenBookQA [19]. The datasets comprise Cloze and Winograd style tasks, alongside multiple-choice question answering. Our evaluation follows the zero-shot approach commonly employed in the language modeling community [12]. As shown in Table 6, XGen-7B achieves comparable performance to the state-of-the-art LLMs of similar sizes.

Table 6: Zero-shot performance on Common Sense Reasoning and Question Answering tasks.

Models	MMLU -wavg	ARC_ch	HellaSwag	Winogrande	TruthfulQA	BoolQ	PiQA	OpenBookQA
XGen-7B	32.1	41.2	74.2	64.9	39.1	74.3	75.5	40.2
LLaMA-7B	32.0	44.8	76.2	69.6	34.0	74.9	78.7	44.2
Falcon-7B	23.9	43.4	76.4	67.2	34.3	73.8	79.4	44.0
MPT-7B	27.4	41.7	76.1	68.6	33.4	74.1	79.1	41.8
OpenLLaMA-7B	28.6	38.7	71.8	67.0	35.2	70.6	76.0	39.0
Redpajama-7B	25.8	39.1	70.3	63.8	33.3	69.3	76.9	40.0
GPT-neox-20B	24.5	41.1	70.5	66.1	31.4	64.9	76.7	38.8
OPT-13B	24.4	35.8	69.9	64.7	33.9	65.0	75.7	39.8
GPT-J-6B	25.7	36.3	66.2	64.5	36.0	65.4	75.4	38.2
Dolly-v2-12B	25.4	39.6	70.8	61.8	34.4	56.3	75.4	39.2
Cerebras-GPT-13B	24.6	32.4	59.4	60.8	39.2	61.1	73.5	35.8
StableLM-alpha-7B	24.4	27.0	40.7	51.5	41.7	59.0	65.8	32.4

Table 7: Natural language to code generation results in pass@1 on the HumanEval benchmark. For OpenLLaMA-7B-v2, we note that Starcoder data occupies 30% of their pre-training data. *Consecutive whitespaces are treated as one, breaking Python syntax. **Model could not generate meaningful code.

Models	pass@1
XGen-7B	14.20
MPT-7B	15.90
OpenLLaMA-7B-v2	14.83
LLaMA-2-7B	13.55
LLaMA-7B	10.38
Redpajama-7B	5.24
OpenLLaMA-7B	0*
Falcon-7B	0**

5.1.2 Code Generation

To evaluate XGen-7B's code generation capability from natural language instructions (i.e., docstrings), we evaluate the model on HumanEval benchmark [4]. HumanEval evaluates LLMs' Python code-writing capabilities at the function level by assessing functional correctness. We report performance using the pass@1 metric [4]. A generated code is considered correct if it passes all the unit tests. Following [4], we set the sampling temperature to 0.2, $p = 0.95$ for top-p sampling, and generate $n = 200$ samples for each problem in the benchmark to report an unbiased pass@1 score. As we can notice in Table 7, our XGen-7B achieves comparable results to state-of-the-art 7B LLMs.

Considering the size and results in both text and code tasks, XGen-7B can be a good general-purpose model that can be served both on standard-sized GPUs (e.g., 16 GB memory) and mobile devices.

5.2 Instruction Model Evaluation

5.2.1 AlpacaEval

AlpacaEval[10] [16] is a newly proposed automated evaluation platform that employs an LLM as an evaluator. It utilizes the AlpacaFarm [11] evaluation dataset, which has been crafted to evaluate a model's ability to understand and follow a wide range of user instructions. The responses generated by the models under evaluation are then contrasted with the reference responses from text-davinci-003 [25], with GPT-4 [24] serving as the evaluator. The win rate against text-davinci-003is employed as the performance metric.

As shown in Table 8, our instruction-tuned model, XGen-7B-Inst$_{\text{wizardLM}}$ (fine-tuned on WizardLM [39]), generally achieves better performance than other models of similar sizes, notably the WizardLM-7B, which uses the same repository of distilled instructions. Our model performs slightly worse than Vicuna-7B-v1.3, which utilizes more ShareGPT data comprising human-authored prompts. The XGen-7B-Inst$_{\text{general}}$ model performs worse than XGen-7B-Inst$_{\text{wizardLM}}$ but still significantly better than text-davinci-003 and other open-source alternatives like Falcon-40B-instruct and MPT-7B-chat.

5.2.2 MT-Bench

Similar to AlpacaEval, MT-Bench[11] [43] is a new benchmark for evaluating LLM-based chat assistants. It also uses an LLM as a judge (e.g., GPT-4) to assess the models on open-ended questions. The model evaluation is performed in two ways:

Single answer grading. In this evaluation setting, the judge LLM assigns a score directly to each of the model-generated responses. As shown in Table 9, XGen-7B-Inst$_{\text{wizardLM}}$ outperforms other models of similar sizes (except Vicuna-7B-v1.3), especially the WizardLM-7B-Inst model which uses a similar instruction set. It even surpasses larger models, such as the Falcon-40B-instruct or MPT-30B-instruct.

[10] `https://tatsu-lab.github.io/alpaca_eval/`

[11] `https://huggingface.co/spaces/lmsys/mt-bench`

Table 8: Results on the AlpacaEval leaderboard [16] with GPT-4 as an evaluator.

Model	**Win Rate vs. text-davinci-003**
GPT-4	95.3
Claude	88.4
ChatGPT	86.1
Vicuna-7B-V1.3	76.8
WizardLM-13B	75.3
Guanaco-65B	71.8
Vicuna-13B	70.4
XGen-7B-Inst$_{\text{wizardLM}}$	68.8
WizardLM-7B	65.2
OAsst-RLHF-LLaMA-33B	66.5
Vicuna-7B	64.4
XGen-7B-Inst$_{\text{general}}$	57.3
text-davinci-003	50.0
Falcon-40B-instruct	45.7
MPT-7B-chat	45.0
Alpaca-farm-PPO-human	41.2
Alpaca-7B	26.5
text-davinci-001	15.2

Pairwise comparison. In this setting, the judge LLM is given a question along with two model responses from two competing models. The judge is tasked to determine which answer is superior, or to declare that both answers are equally good. From the results in Table 10, we see that here also XGen-7B-Inst models outperform other models of similar sizes and they surpass some larger models.

5.3 Long Sequence Tasks

In addition to public benchmarks AlpacaEval and MT-Bench, we also evaluate XGen-7B and other competitive open source models on long sequence modeling tasks.

5.3.1 Long-form QA

In order to evaluate the reasoning capabilities of open source LLMs on long context, we design a long-form QA task in-house with two settings. Given a long input document, (1) we first prompt ChatGPT (GPT-3.5-turbo) to generate questions with explicit instructions such that answers are not directly retrievable from the context with few words. We call this setting **QG-passage**. (2) In order to generate more abstract questions that would require synthesizing different elements from different parts of the input document, we first summarize the document and then generate questions on the summary using ChatGPT similar to (1). We call this setting **QG-summary**. We provide examples of the prompts in Appendix A.1.

Next, we prompt the models to answer the questions generated from ChatGPT on the two settings mentioned above. Note that we know the ground-truth answers in the two settings. We set a maximum of 512 tokens for generation. We use GPT-4 for evaluating the responses on the generated answers and rate them on a scale of 0-3 for the following dimensions: coherence, relevance, and accuracy.

As shown in Table 11, we find that XGen-7B-Inst$_{\text{general}}$ outperforms all the other models compared. Specifically, we find that the rates for XGen-7B-Inst models are higher for generated responses in terms of coherence and relevance. In general, we find that questions generated from summary are often more difficult to generate response which shows the difficulty of the overall setting (Table 12). These improvements can be partially attributed to XGen's long-sequence modeling capability.

5.3.2 Dialogue Summarization

In order to evaluate the long dialogue understanding and summarization capabilities, we perform experiments on three dialogue summarization tasks: AMI meeting summarization [3], screenplay

Table 9: Evaluation on MT Bench [43] – Single answer grading by GPT-4.

Model	Score
GPT-4	8.99
ChatGPT(GPT-3.5-turbo)	7.94
Claude-v1	7.90
Claude-instant-v1	7.85
Vicuna-33B-v1.3	7.12
WizardLM-30B	7.01
Guanaco-33B	6.53
Tulu-30B	6.43
Guanaco-65B	6.41
OAsst-SFT-7-LLaMA-30B	6.41
PaLM-2-chat-bison-001	6.40
MPT-30B-chat	6.39
Vicuna-13B-v1.3	6.39
WizardLM-13B	6.35
Vicuna-7B-v1.3	6.00
Baize-v2-13B	5.75
XGen-7B-Inst$_{\text{wizardLM}}$	5.69
XGen-7B-Inst$_{\text{general}}$	5.54
Nous-Hermes-13B	5.51
MPT-7B-chat	5.42
GPT4All-13B-snoozy	5.41
Koala-13B	5.35
WizardLM-7B	5.29
MPT-30B-instruct	5.22
Falcon-40B-instruct	5.17
H2OGPT-OAsst-Open-LLaMA-13B	4.63
Alpaca-13B	4.53
ChatGLM-6B	4.50
OAsst-SFT-4-pythia-12B	4.32
RWKV-4-raven-14B	3.98
Dolly-v2-12B	3.28
Fastchat-T5-3B	3.04
StableLM-tuned-alpha-7B	2.75
LLaMA-13B	2.61

summarization from ForeverDreaming (FD) and TVMegaSite (TMS) datasets [5]. The average source lengths for these datasets are 5570, 6466, and 7653 tokens, respectively.

For evaluation shown in Table 13, we focus on samples with lengths less than 8K and consider the same instruction-tuned models as above. It is worth noting that both MPT-7B-inst and Alpaca-7B models performed poorly in this setting when input truncation was not applied. In contrast, our model (XGen-7B-Inst) achieved the highest ROUGE scores across all metrics.

6 Carbon Footprint

To estimate the energy consumption and the resulting emission of carbon dioxide for training XGen-7B, we follow [38]. Specifically, we compute Mega-watt-hour (MWh) as follows:

$$\begin{aligned} \text{MWh} &= \text{TPU-hours} \times (\text{TPU power consumption}) \times \text{PUE} && (1) \\ &= 270,336 \times 192 \times 1.10 && (2) \\ &= 57 && (3) \end{aligned}$$

where we set the Power Usage Effectiveness (PUE) to 1.10 following the standard. The resulting carbon emission depends on the data center location. For XGen-7B, this amounts to: tCO_2eq = MWh $(57) \times 0.079 = 4.5$. In Figure 3, we show this in comparison with other LLMs.

Table 10: MT Bench Evaluation [43] – Pairwise Comparison by GPT-4.

Model	Win	Loss	Tie	Win Rate	Loss Rate	Win Rate Adjusted
GPT-4	111	7	42	69.4	43.8	82.5
Claude-v1	75	27	58	46.9	16.9	65.0
Vicuna-33B-v1.3	70	42	48	43.8	26.3	58.8
Claude-instant-v1	64	40	56	40.0	25.0	57.5
WizardLM-30B	37	63	60	23.1	39.4	41.9
Guanaco-33B	42	72	46	26.3	45.0	40.6
Guanaco-65B	38	68	54	23.8	42.5	40.6
Vicuna-13B-v1.3	33	73	54	20.6	45.6	37.5
MPT-30B-chat	29	78	53	18.1	48.8	34.7
Vicuna-7B-v1.3	60	165	95	18.8	51.6	33.6
WizardLM-13B	27	81	52	16.9	50.6	33.1
Tulu-30B	29	92	39	18.1	57.5	30.3
OAsst-SFT-7-LLaMA-30B	23	88	49	14.4	55.0	29.7
XGen-7B-Inst$_{\text{wizardLM}}$	22	91	47	13.8	56.9	28.4
Baize-v2-13B	21	101	38	13.1	63.1	25.0
PaLM-2-chat-bison-001	18	102	40	11.3	63.8	23.8
XGen-7B-Inst$_{\text{general}}$	17	108	35	10.6	67.5	21.6
Nous-Hermes-13B	12	104	44	7.5	65.0	21.3
GPT4All-13B-snoozy	14	108	38	8.8	67.5	20.6
MPT-7B-chat	18	214	88	5.6	66.9	19.4
H2OGPT-OAsst-Open-LLaMA-13B	19	118	23	11.9	73.8	19.1
Koala-13B	10	110	40	6.3	68.8	18.8
Falcon-40B-instruct	10	116	34	6.3	72.5	16.9
MPT-30B-instruct	7	120	33	4.4	75.0	14.7
ChatGLM-6B	6	124	30	3.8	77.5	13.1
OAsst-SFT-4-pythia-12B	8	128	24	5.0	80.0	12.5
RWKV-4-raven-14B	6	128	26	3.8	80.0	11.9
Alpaca-13B	13	265	42	4.1	82.8	10.6
Fastchat-T5-3B	5	132	23	3.1	82.5	10.3
Dolly-v2-12B	5	138	17	3.1	86.3	8.4

Table 11: Overall performance of different models based on GPT-4 evaluation on long-form QA. The table shows individual and average ratings across all metrics: coherence, relevance and accuracy.

Model	Coherence	Relevance	Accuracy	Avg.
XGen-7B-Inst$_{\text{general}}$	**2.81**	**2.72**	**2.70**	**2.74**
Vicuna-7B-v1.3	2.77	2.64	2.58	2.66
XGen-7B-Inst$_{\text{wizardLM}}$	2.78	2.68	2.50	2.65
WizardLM-7B	2.79	2.74	2.40	2.63
MPT-7B-instruct	2.55	2.48	2.30	2.43
Falcon-7B-instruct	2.28	2.22	1.75	2.08
Alpaca-7B	1.65	1.91	1.58	1.71

7 Note on Potential Risks

Finally, despite our efforts in addressing the risks of bias, toxicity, and hallucinations both in pre-training and fine-tuning stages, like other LLMs, XGen-7B models are not free from such limitations. We hope our open-sourced codebase will help other researchers better understand these challenges and improve on these key limitations for making AI beneficial for everyone.

8 Conclusion

In this report, we have presented our newly developed XGen-7B models that support up to 8K tokens as input context. We described its effective stage-wise pre-training process with different sequence lengths (2K $\rightarrow$ 4K $\rightarrow$ 8K) and data mixtures (mostly text $\rightarrow$ 50% text - 50% code). We have shown

Table 12: Performance breakdown of different models in the two settings based on GPT-4 evaluation. The table shows average ratings across all metrics for questions generated from passage (QG-passage) and summary (QG-summary).

Model	QG-passage	QG-summary
XGen-7B-Inst$_{\text{general}}$	**2.79**	**2.68**
Vicuna-7B-v1.3	2.71	2.61
XGen-7B-Inst$_{\text{wizardLM}}$	2.71	2.60
WizardLM-7B	2.71	2.55
MPT-7B-instruct	2.50	2.35
Falcon-7B-instruct	2.22	1.95
Alpaca-7B	2.04	1.64

Table 13: ROUGE scores of different models on long dialogue summarization task.

Model	AMI			FD			TMS		
	R-1	R-2	R-L	R-1	R-2	R-L	R-1	R-2	R-L
XGen-7B-Inst$_{\text{general}}$	**31.34**	**8.25**	**17.00**	**29.34**	**5.39**	**16.43**	**26.39**	**3.94**	**13.71**
XGen-7B-Inst$_{\text{wizardLM}}$	25.56	6.71	16.84	8.97	0.90	5.49	19.15	1.86	9.53
Vicuna-7B-v1.3	14.23	2.01	9.05	16.49	1.00	9.99	17.06	1.49	8.85
Falcon-7B-instruct	14.89	1.97	9.28	18.90	1.80	9.37	18.90	1.80	9.37
MPT-7B-instruct	11.95	1.88	8.10	14.27	1.40	8.89	19.80	2.39	10.23
Alpaca-7B	9.69	1.77	6.43	16.26	1.56	10.66	12.26	1.15	7.30
WizardLM-7B	18.97	2.65	10.32	14.13	1.11	8.07	19.16	1.87	9.51

that the resulting model achieves comparable or better results on standard text and code generation benchmarks compared to state-of-the-art open-source LLMs.

We have also described the finetuning process of the XGen-7B model on two different public-domain instructional datasets, creating the XGen-7B-Inst counterparts. The results on two popular benchmarks show that our models often outperform existing models of similar sizes and sometimes even much larger models. We then evaluated the models on long sequence modeling tasks, which validates the superiority of our 8K-sequence model over the existing 2K-sequence LLMs. Finally, we hope that the open-sourcing of our models will contribute to open science in understanding the strengths and limitations of LLMs and will have significant impacts on business and commerce.

9 Author Contribution

Pre-training Model Erik Nijkamp (*lead*), Hiroaki Hayashi, Tian Xie, Chen Xing

Pre-training Data Tian Xie (*lead*), Hiroaki Hayashi, Lidiya Murakhovs'ka

Evaluation Congying Xia (*lead*), Tian Xie, Erik Nijkamp, Rui Meng, Hiroaki Hayashi, Wojciech Kryściński, Ye Liu, Lifu Tu, Meghana Bhat

Instruction Tuning (Model) Bo Pang (*lead*), Chen Xing

Instruction Tuning (Tool & Data) Jesse Vig, Semih Yavuz, Chen Xing, Philippe Laban, Ben Krause, Senthil Purushwalkam, Tong Niu, Wojciech Kryściński, Lidiya Murakhovs'ka, Prafulla Kumar Choubey, Alex Fabbri, Ye Liu, Rui Meng, Lifu Tu, Meghana Bhat

Project Coordination Caiming Xiong (*co-lead*), Shafiq Joty (*co-lead*), Yingbo Zhou (*co-lead*), Chien-Sheng Wu, Silvio Savarese

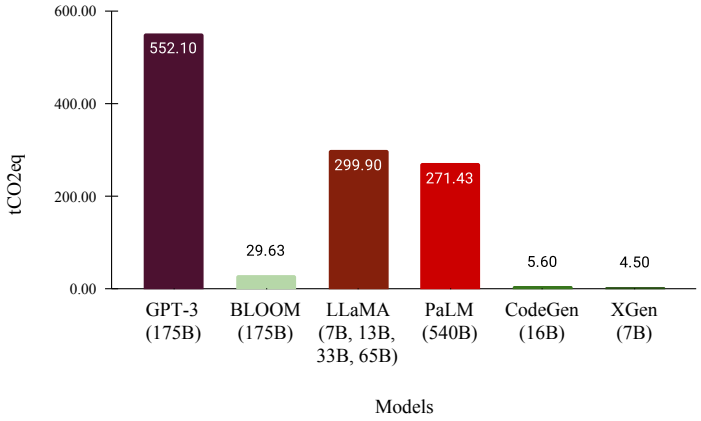

Figure 3: Carbon emission of different models.

References

[1] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 7432–7439, 2020.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Jean Carletta, Simone Ashby, Sebastien Bourban, Mike Flynn, Mael Guillemot, Thomas Hain, Jaroslav Kadlec, Vasilis Karaiskos, Wessel Kraaij, Melissa Kronenthal, Guillaume Lathoud, Mike Lincoln, Agnes Lisowska, Iain McCowan, Wilfried Post, Dennis Reidsma, and Pierre Wellner. The ami meeting corpus: A pre-announcement. In *Proceedings of the Second International Conference on Machine Learning for Multimodal Interaction*, MLMI'05, page 28–39, Berlin, Heidelberg, 2005. Springer-Verlag.

[4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[5] Mingda Chen, Zewei Chu, Sam Wiseman, and Kevin Gimpel. SummScreen: A dataset for abstractive screenplay summarization. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8602–8615, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[6] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. Palm: Scaling language modeling with pathways. *arXiv preprint arXiv:2204.02311*, 2022.

[7] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. *arXiv preprint arXiv:1905.10044*, 2019.

[8] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.

[9] Together Computer. Redpajama: An open source recipe to reproduce llama training dataset, April 2023.

[10] Mike Conover, Matt Hayes, Ankit Mathur, Jianwei Xie, Jun Wan, Sam Shah, Ali Ghodsi, Patrick Wendell, Matei Zaharia, and Reynold Xin. Free dolly: Introducing the world's first truly open instruction-tuned llm, 2023.

[11] Yann Dubois, Xuechen Li, Rohan Taori, Tianyi Zhang, Ishaan Gulrajani, Jimmy Ba, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Alpacafarm: A simulation framework for methods that learn from human feedback, 2023.

[12] Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, September 2021.

[13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring Massive Multitask Language Understanding. In *International Conference on Learning Representations*, 2021.

[14] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katherine Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack William Rae, and Laurent Sifre. An empirical analysis of compute-optimal large language model training. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.

[15] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Gontier, Nicholas Meade, Armel Zebaze, Ming-Ho Yee, Logesh Kumar Umapathi, Jian Zhu, Benjamin Lipkin, Muhtasham Oblokulov, Zhiruo Wang, Rudra Murthy, Jason Stillerman, Siva Sankalp Patel, Dmitry Abulkhanov, Marco Zocca, Manan Dey, Zhihan Zhang, Nour Fahmy, Urvashi Bhattacharyya, Wenhao Yu, Swayam Singh, Sasha Luccioni, Paulo Villegas, Maxim Kunakov, Fedor Zhdanov, Manuel Romero, Tony Lee, Nadav Timor, Jennifer Ding, Claire Schlesinger, Hailey Schoelkopf, Jan Ebert, Tri Dao, Mayank Mishra, Alex Gu, Jennifer Robinson, Carolyn Jane Anderson, Brendan Dolan-Gavitt, Danish Contractor, Siva Reddy, Daniel Fried, Dzmitry Bahdanau, Yacine Jernite, Carlos Muñoz Ferrandis, Sean Hughes, Thomas Wolf, Arjun Guha, Leandro von Werra, and Harm de Vries. Starcoder: may the source be with you!, 2023.

[16] Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Alpacaeval: An automatic evaluator of instruction-following models. `https://github.com/tatsu-lab/alpaca_eval`, 2023.

[17] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*, 2021.

[18] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, et al. The flan collection: Designing data and methods for effective instruction tuning. *arXiv preprint arXiv:2301.13688*, 2023.

[19] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. *arXiv preprint arXiv:1809.02789*, 2018.

[20] Igor Molybog, Peter Albert, Moya Chen, Zachary DeVito, David Esiobu, Naman Goyal, Punit Singh Koura, Sharan Narang, Andrew Poulton, Ruan Silva, et al. A theory on adam instability in large-scale machine learning. *arXiv preprint arXiv:2304.09871*, 2023.

[21] Ann E Nicholson, Kevin B Korb, Erik P Nyberg, Michael Wybrow, Ingrid Zukerman, Steven Mascaro, Shreshth Thakur, Abraham Oshni Alvandi, Jeff Riley, Ross Pearson, et al. Bard: A structured technique for group elicitation of bayesian networks to support analytic reasoning. *arXiv preprint arXiv:2003.01207*, 2020.

[22] Erik Nijkamp. Jaxformer: A minimal library for training llms on tpu. `https://github.com/salesforce/jaxformer`, 2022.

[23] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022.

[24] OpenAI. Gpt-4 technical report, 2023.

[25] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[26] Ofir Press, Noah Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.

[27] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.

[28] Victor Sanh, Albert Webson, Colin Raffel, Stephen H Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207*, 2021.

[29] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[30] Uri Shaham, Elad Segal, Maor Ivgi, Avia Efrat, Ori Yoran, Adi Haviv, Ankit Gupta, Wenhan Xiong, Mor Geva, Jonathan Berant, and Omer Levy. SCROLLS: Standardized CompaRison over long language sequences. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 12007–12021, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics.

[31] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

[32] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model. `https://github.com/tatsu-lab/stanford_alpaca`, 2023.

[33] Yi Tay, Mostafa Dehghani, Samira Abnar, Yikang Shen, Dara Bahri, Philip Pham, Jinfeng Rao, Liu Yang, Sebastian Ruder, and Donald Metzler. Long range arena : A benchmark for efficient transformers. In *International Conference on Learning Representations*, 2021.

[34] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[35] Ben Wang and Aran Komatsuzaki. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. `https://github.com/kingoflolz/mesh-transformer-jax`, May 2021.

[36] Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Anjana Arunkumar, Arjun Ashok, Arut Selvan Dhanasekaran, Atharva Naik, David Stap, et al. Super-naturalinstructions:generalization via declarative instructions on 1600+ tasks. In *EMNLP*, 2022.

[37] Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

[38] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga, Jinshi Huang, Charles Bai, Michael Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin Lee, Bugra Akyildiz, Maximilian Balandat, Joe Spisak, Ravi Jain, Mike Rabbat, and Kim Hazelwood. Sustainable ai: Environmental implications, challenges and opportunities. In D. Marculescu, Y. Chi, and C. Wu, editors, *Proceedings of Machine Learning and Systems*, volume 4, pages 795–813, 2022.

[39] Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. Wizardlm: Empowering large language models to follow complex instructions. *arXiv preprint arXiv:2304.12244*, 2023.

[40] Canwen Xu, Daya Guo, Nan Duan, and Julian McAuley. Baize: An open-source chat model with parameter-efficient tuning on self-chat data. *arXiv preprint arXiv:2304.01196*, 2023.

[41] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*, 2019.

[42] Biao Zhang and Rico Sennrich. Root mean square layer normalization. *Advances in Neural Information Processing Systems*, 32, 2019.

[43] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

A Appendix

A.1 Long form QA - Prompts used for Question Generation

We formulate our question generation method as a two-step process: (1) Summarization and (2) Question generation from summary. In the first step, we design a prompt for generating a summary as shown below:

Summarize the paragraphs below in the context of {title} in {domain}.

In the next step, we ask ChatGPT to generate questions from summary as shown below:

Using the context below, come up with follow-up questions such that answers are beyond few words or a couple of phrases. Rank the generated questions in the order of decreasing complexity to answer and display only the top 3. {context}

To demonstrate the usefulness of our question generation process, we also establish a baseline with the same instructions where questions are directly generated from the passage. The prompt used for the baseline is:

Using the context below, come up with follow-up questions such that answers are beyond few words or a couple of phrases. Rank the generated questions in the order of decreasing complexity to answer and display only the top 3. {context}